

Best practical architecture for তোমাদের

আমি recommend করবো:

FastAPI / Express.js

+

Rule-based reasoning engine

+

Optional LLM for summary only

+

Safety filter

+

Docker

+

Render/Railway deployment

LLM দিয়ে পুরো decision নেওয়ানো risky। কারণ LLM ভুল enum দিতে পারে, refund promise করতে পারে, অথবা prompt injection-এ পড়ে যেতে পারে।

Better architecture:

Rule-based engine → final decision

LLM → only language polish, optional

Safety guardrail → final check

Full flow example

Input:

```
{  
  "ticket_id": "TKT-007",  
  "complaint": "Someone called me and asked for my OTP to get cashback.",  
  "transaction_history": []  
}
```

Architecture flow:

Request Validator

→ valid

Preprocessor

→ keywords: OTP, called, cashback

Case Detector

→ phishing_or_social_engineering

Transaction Matcher

→ no transaction needed

Evidence Verdict

→ insufficient_data

Department Router

→ fraud_risk

Severity

→ critical

Safety Guardrail

→ add warning, never ask OTP

Response Builder

→ final JSON

Output:

```
{
  "ticket_id": "TKT-007",
  "relevant_transaction_id": null,
  "evidence_verdict": "insufficient_data",
  "case_type": "phishing_or_social_engineering",
  "severity": "critical",
  "department": "fraud_risk",
  "agent_summary": "Customer reports a suspicious call asking for OTP related to cashback.",
  "recommended_next_action": "Escalate to fraud risk team and advise the customer to use only official support channels.",
  "customer_reply": "This may be a suspicious attempt. Please do not share your PIN, OTP, password, or any sensitive information with anyone. Contact only official support channels.",
  "human_review_required": true,
  "confidence": 0.95,
  "reason_codes": ["otp_request", "phishing_risk"]
}
```

Node/Express হল:

queuestorm-investigator/

src/

server.js

routes/

health.route.js

analyze.route.js

schemas/

ticket.schema.js

core/

preprocessor.js

caseDetector.js

transactionMatcher.js

evidenceEngine.js

departmentRouter.js

severityEngine.js

safetyGuardrail.js

responseBuilder.js

utils/

amountParser.js

textUtils.js

tests/

sample_outputs/

package.json

Dockerfile

README.md

.env.example

Final architecture in one diagram

Judge Harness

|

| GET /health

| POST /analyze-ticket

v

API Server

|

v

Input Validation

|

v

Complaint Text Analyzer

|

v

Transaction Matcher

|

v

Evidence Reasoning Engine

|

v

Case Type Classifier

|

v

Department + Severity Router

|

v

Safety Guardrail

|

v

Response Builder

|

v

Structured JSON Response